Informe de Laboratorio: Estructura de Computadores

Nombre de los estudiantes:

- Johan Sebastian Castillo Mazo
- Ivan Camilo Parra Quintero **Fecha:** 27/02/2026
  Asignatura: Estructura de Computadores

Enlace del repositorio en GitHub:

- Github Camilo Parra

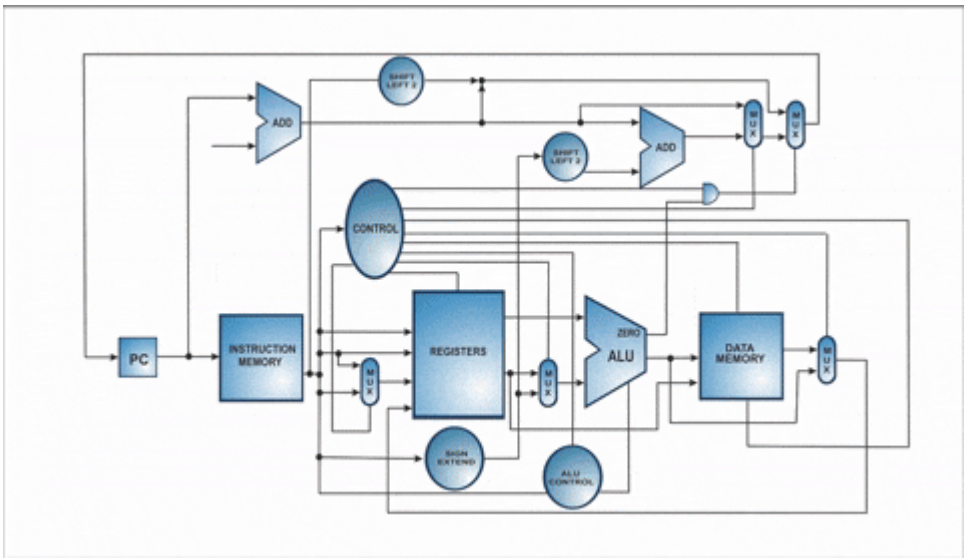
---

1. Análisis del Código Base

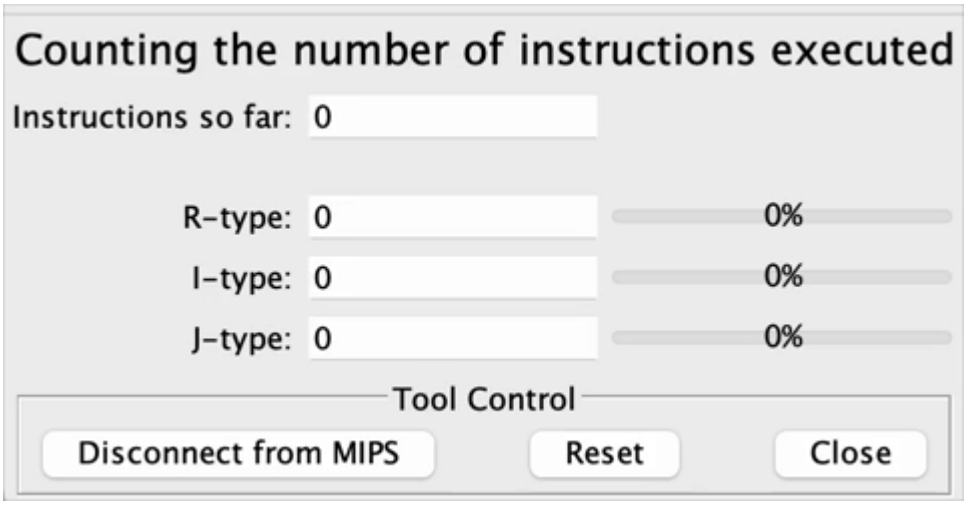
1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

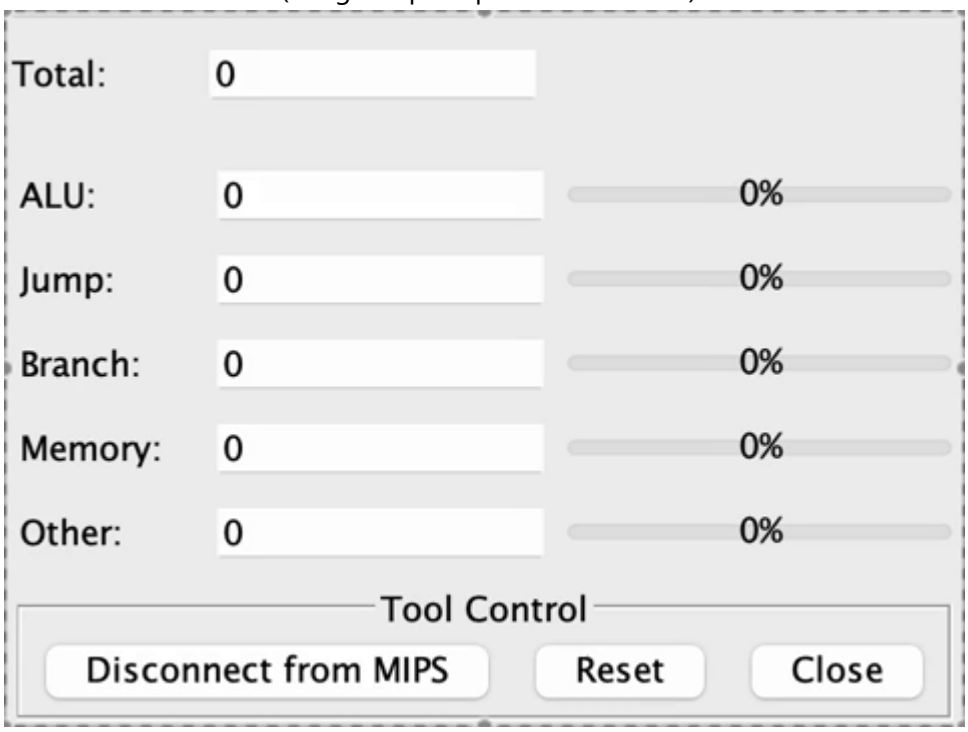
- **MIPS X-Ray** (Ventana con el Datapath animado)

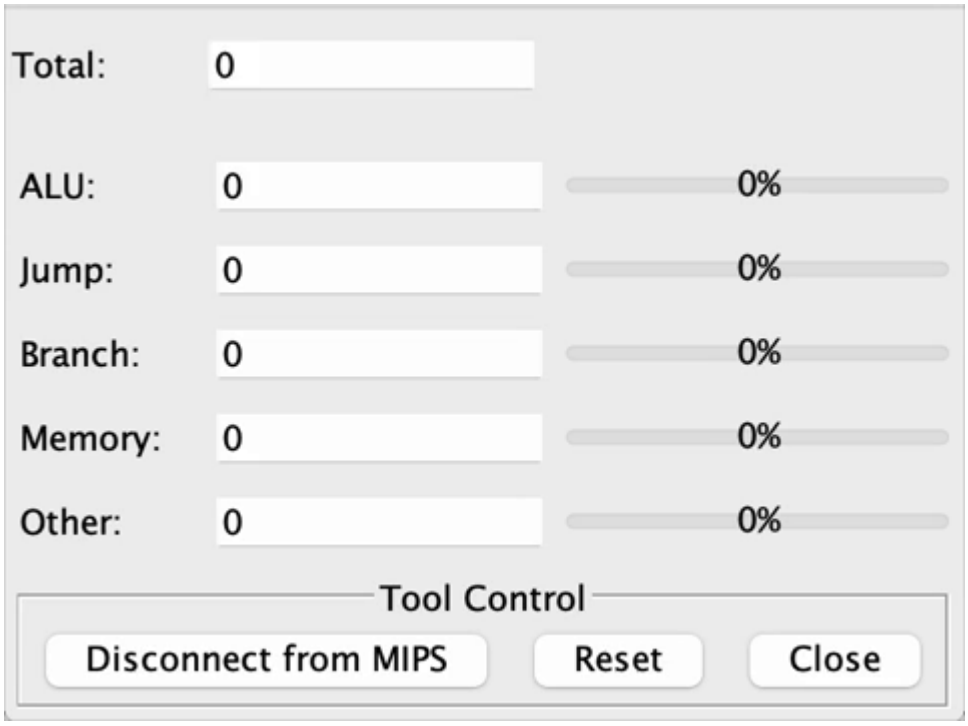


- **Instruction Counter** (Contador de instrucciones totales).



- **Instruction Statistics** (Desglose por tipo de instrucción).





1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo	Ciclos de Parada (Con Forwarding)	Ciclos de Parada (Sin Forwarding)
`sll $t4, $t3, 2`	`addu $t5, $s0, $t4`	Riesgo de Datos (RAW en `$t4`)	0	2
`addu $t5, $s0, $t4`	`lw $t6, 0($t5)`	Riesgo de Datos (RAW en `$t5`)	0	2

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo	Ciclos de Parada (Con Forwarding)	Ciclos de Parada (Sin Forwarding)
`lw $t6, 0($t5)`	`mul $t7, $t6, $t0`	**Riesgo de Datos (Load-Use en `$t6`)**	**1 (Inevitable)**	**2**
`mul $t7, $t6, $t0`	`addu $t8, $t7, $t1`	Riesgo de Datos (RAW en `$t7`)	0	2
`addu $t9, $s1, $t4`	`sw $t8, 0($t9)`	Riesgo de Datos (RAW en `$t9`)	0	2
`addu $t8, $t7, $t1`	`sw $t8, 0($t9)`	Riesgo de Datos (RAW en `$t8`)	0	1*
`beq $t3, $t2, fin`	Siguiente instrucción	Riesgo de Control (Branch)	1	1
`j loop`	Siguiente instrucción	Riesgo de Control (Jump)	1	1

Aqui podemos ver como el rendimiento del procesador cambia drásticamente dependiendo de si este cuenta o no con la técnica de **Forwarding** . Esta tecnología de hardware permite que el resultado de una operación anterior se pase directamente desde la etapa de Ejecución (EX) a la entrada de la ALU en el nuevo ciclo. Sin esta técnica, la instrucción dependiente tendría que detenerse (generando ciclos de parada) hasta que el dato termine de escribirse físicamente en el banco de registros durante la etapa final de Write-Back (WB) para recién poder leerlo.

1.2. Estadísticas y Análisis Teórico

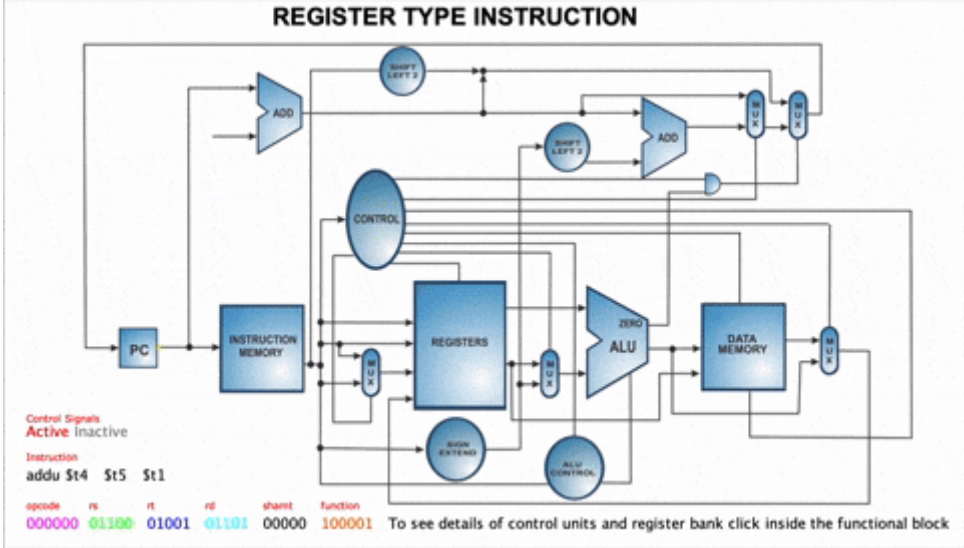
Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

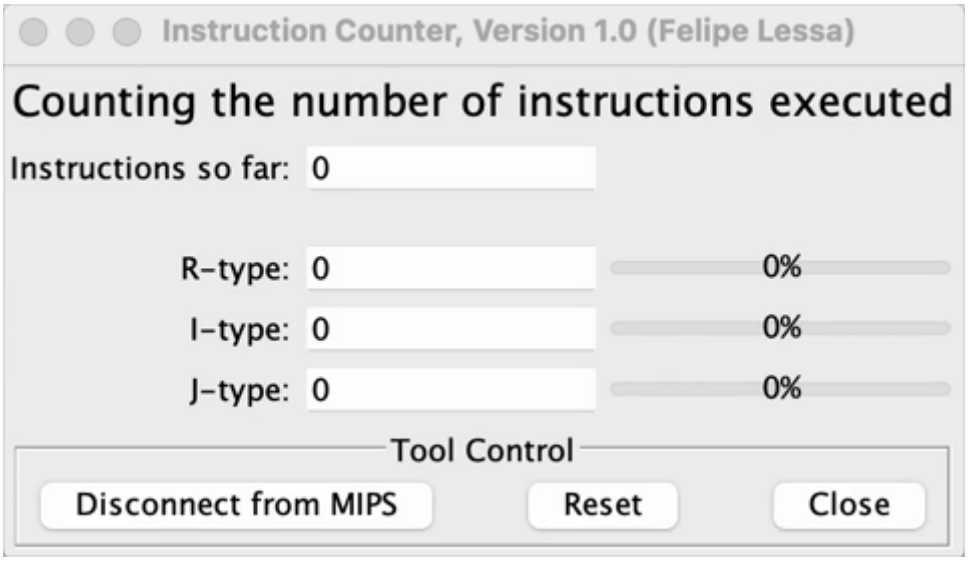
Métrica	Código Base	Código Optimizado
Instrucciones Totales (según MARS)	94	58
Stalls (Paradas) por iteración	1	0
Total de Stalls (8 iteraciones)	8	0
Ciclos Totales Estimados (Inst + Stalls)	102	58
CPI Estimado (Ciclos / Inst)	1.09	1.00

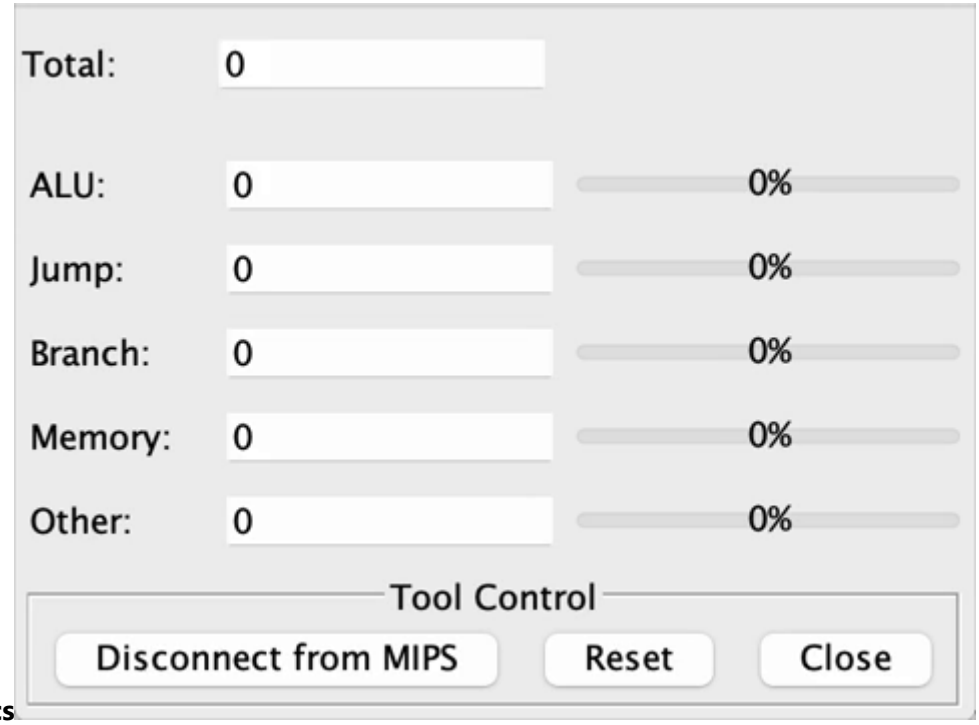
2. Optimización Propuesta

2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

- **MIPS X-Ray** 

- **Instruction Counter** 

- **Instruction Statistics** 

2.2. Código Optimizado

Aqui se detalla la optimización integral del código, la cual incluye no solo el desenrollado del bucle (*loop unrolling*), sino también el pre-cálculo del límite de iteraciones y la implementación de aritmética de punteros

```
.data
    vector_x: .word 1, 2, 3, 4, 5, 6, 7, 8
    vector_y: .space 32
    const_a:  .word 3
    const_b:  .word 5
    tamano:   .word 8

.text
.globl main
main:
    # --- Inicialización de punteros y constantes ---
    la $s0, vector_x          # $s0 = puntero actual a X[i]
    la $s1, vector_y          # $s1 = puntero actual a Y[i]
    lw $t0, const_a           # $t0 = A = 3
    lw $t1, const_b           # $t1 = B = 5
    lw $t2, tamano            # $t2 = 8 (número de elementos)

    # Pre-calculamos el límite FUERA del loop (evita cálculo repetido)
    sll $t2, $t2, 2           # $t2 = 8 * 4 = 32 bytes
    addu $s2, $s0, $t2        # $s2 = &X[8] = dirección de parada

    # LOOP UNROLLING: procesamos 2 elementos por iteración
    # Ventaja: las instrucciones del elemento i+1 llenan los huecos
    #          de dependencia del elemento i, eliminando stalls.
    # Requisito: el vector debe tener número par de elementos (aquí 8, OK).
loop:
    # === CARGA DE X[i] y X[i+1] ===
    lw $t6, 0($s0)            # $t6 = X[i]
    lw $t3, 4($s0)            # $t3 = X[i+1]  ← ocupa el Load-Use slot de $t6

    # === MULTIPLICACIÓN ===
    mul $t7, $t6, $t0         # $t7 = X[i] * A      | $t6 ya disponible (2 ciclos
pasaron)
    mul $t4, $t3, $t0         # $t4 = X[i+1] * A    | llena el RAW slot de
mul→addu de i

    # === SUMA CON B ===
    addu $t8, $t7, $t1        # $t8 = X[i]*A + B
    addu $t5, $t4, $t1        # $t5 = X[i+1]*A + B  | llena el RAW slot de
mul→addu de i+1

    # === ALMACENAMIENTO ===
    sw $t8, 0($s1)            # Y[i]   = $t8
    sw $t5, 4($s1)            # Y[i+1] = $t5

    # === AVANCE DE PUNTEROS (2 elementos = 8 bytes) ===
    addiu $s0, $s0, 8         # X += 2 posiciones
    addiu $s1, $s1, 8         # Y += 2 posiciones

    # === CONDICIÓN DE SALIDA ===
    bne $s0, $s2, loop        # Si no llegamos al límite, repetir
```

```
fin:
    li $v0, 10
    syscall
```

2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

La instrucción que movimos al hueco fue `lw $t3, 4($s0)`, que carga el elemento X[i+1] del vector. En el código original, `mul $t7, $t6, $t0` aparecía inmediatamente después de `lw $t6, 0($s0)`, generando un **Load-Use Hazard**: el dato de `$t6` solo está disponible al finalizar la etapa MEM del `lw`, pero `mul` lo necesita al inicio de su etapa EX, provocando que el hardware inserte una burbuja automáticamente, perdiendo un ciclo útil.

Para eliminarlo, apliqué **reordenación de instrucciones por software**: busqué una instrucción independiente más adelante en el código y la moví al hueco. La instrucción `lw $t3, 4($s0)` cumple esta condición porque no lee ni escribe `$t6`, por lo que no genera nuevas dependencias. Al colocarla entre el primer `lw` y el `mul`, el pipeline tiene el ciclo necesario para que `$t6` esté disponible cuando `mul` lo requiere, convirtiendo un ciclo muerto en trabajo útil.

Como beneficio adicional, el segundo `mul $t4, $t3, $t0` ocupa el hueco de dependencia RAW entre `mul $t7` y `addu $t8`, eliminando también ese stall potencial. De esta forma, las instrucciones del elemento i+1 actúan sistemáticamente como relleno para los huecos del elemento i.

Finalmente, para potenciar esta reordenación, se eliminó el cálculo tradicional de índices en favor de la **aritmética de punteros**. En el código base, recalcular la dirección de memoria en cada iteración consumía instrucciones adicionales (`sll` y `addu`) para multiplicar el índice y sumarlo a la dirección base. Al pre calcular la dirección límite de memoria fuera del bucle y actualizar los punteros directamente sumando bytes (`addiu $s0, $s0, 8`), se eliminó el sobrecosto computacional del contador. Esto redujo drásticamente el número total de instrucciones ejecutadas y dejó el bucle lo suficientemente limpio como para que el desenrollado (loop unrolling) fuera increiblemente eficiente! alcanzando la perfección en un ambiente ideal.

3. Comparativa de Resultados

Métrica	Código Base	Código Optimizado	Mejora (%)
Ciclos Totales	102	58	43.1%
Stalls (Paradas)	8	0	100%
CPI	1.09	1.00	8.3%

---

4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todas las paradas?

El impacto de la segmentación en el diseño a bajo nivel es increiblemente profundo. Tras analizar y optimizar el código en el simulador MARS, entendimos que programar en ensamblador exige conocer la arquitectura

física del procesador. La segmentación obliga al programador a estructurar el código estratégicamente para mantener ocupadas todas las etapas del pipeline y evitar tiempos muertos. Como ingenieros, a menudo el enfoque inicial se centra en la resolución lógica del problema, dejando de lado la eficiencia. Sin embargo, el diseño a bajo nivel demuestra que comprender y aprovechar los recursos de hardware disponibles puede resultar en mejoras de rendimiento enormes, reduciendo drásticamente los tiempos de ejecución con modificaciones que en principio parecian menores o triviales.

Respecto a la eliminación de las paradas, creemos que no siempre es posible erradicarlas por completo. Aunque en la optimización de este laboratorio logramos un escenario ideal, en la práctica esto depende intrínsecamente de la naturaleza y complejidad del algoritmo. Si un programa tiene una alta densidad de dependencias secuenciales, múltiples saltos condicionales (Control Hazards) o accesos a memoria donde no hay suficientes instrucciones independientes para reordenar, la generación de burbujas será inevitable. No obstante, el objetivo en la ingeniería de software siempre debe ser aplicar cualquier técnica posible para mitigar estas paradas hasta el límite que la lógica del algoritmo permita.